SECTION 2 | LECTURE 5

기준 사례 소개 & 샘플 코드 흐름 살펴보기

Part 1 전체에서 비용, 품질, 리스크, 흐름을 분해할 공통 baseline 애플리케이션을 통해 미리 작성된 기준 코드의 흐름을 파악해봅시다

PART 1 LEARNING ROADMAP

LECTURE 5 (TODAY)	LECTURE 6 - 9	LECTURE 10 - 16
baseline 소개	**문제 정의 & 관측**	**통제 패턴 적용**
공통 이해 & 클래스 매핑	통제 4축 문제지점 하이라이트	Evaluator / Routing / Optimizer

구현을 따라서 타이핑 하는 수업이 아니라, 작성된 기준 코드의 흐름을 읽고 통제 포인트를 해부하는 수업입니다.

기준 사례 앱 소개: Incident Responder는 무엇을 하는가

INPUT UNSTRUCTURED INCIDENT

장애 상황 설명 입력

```
"payment-api에서 500 에러가 급증하고 있다.
DB 커넥션 타임아웃 지표가 함께 상승 중이며,
원인 분석과 긴급 조치 가이드가 필요함"
```

비정형 예외 메시지 및 운영 텍스트 전달

→

OUTPUT STRUCTURED BRIEF

구조화 대응 브리프 생성

- **상황 요약:** 결제 서비스 커넥션 풀 고갈 기반 500 장애
- **추정 원인:** DB 처리 지연으로 인한 스레드 병목 현상
- **추천 액션:** Read Replica 트래픽 분산 및 스레드풀 임시 증설
- **에스컬레이션:** 인프라 DBA 팀 1차 긴급 호출 지정

Incident in → Actionable response out

이 수업에서 코드를 보는 방식

RECOMMENDED APPROACH

DO : Flow Reading (흐름 중심 분석)

전체 파이프라인의 입출력 연결과 클래스간 역할 분담 파악

- **전체 흐름 추적:** 요청 수신부터 AI 호출 및 응답 반환까지 흐름 파악
- **클래스 역할 매핑:** Controller, Service, Port가 담당하는 책임 경계 구분
- **기준 사례 고정:** Part 1 전체에서 함께 해부할 뼈대로 머릿속에 안착

"구조와 흐름을 맵처럼 읽는 시간"

AVOID FOR NOW

DON'T : Detail Optimization (지나친 디테일)

세부 구현 코드 한 줄 단위의 완성도 단정 및 리팩터링 시도

- **세부 구현 집착:** 로직 한 줄이나 텍스트 처리 방식에 빠져 전체 흐름 놓치기
- **성급한 리팩터링:** 통제 문제점이 무엇인지 정의되기도 전에 코드 수정하기
- **완성도 단정:** 데모가 동작한다는 이유로 운영 준비가 끝났다고 판단하기

"세부 문제는 다음 수업에서 통제 4축으로 해부합니다"

앱의 전체 흐름: 입력에서 추천 액션까지

단순 One-shot 질의응답이 아닌, **컨텍스트 수집과 단계별 연산을 경유하는 6단계 파이프라인**

STEP 01
Incident 요청 수신
운영자 장애 브리프 전달

STEP 02
관련 정보 조회
Runbook / History / Policy

STEP 03
분석 Prompt 생성
컨텍스트 바인딩 및 템플릿화

STEP 04
AI 상황 분석
Spring AI ChatClient 연동

STEP 05
액션 추천 생성
대응 가이드 & 런북 매핑

STEP 06
응답 반환
구조화 DTO 형태로 최종 출력

핵심 포인트: AI 호출 이전에 이미 외부 도메인 정보를 수집하고 조합하는 파이프라인 단계를 거칩니다.

흐름과 클래스 매핑: 파이프라인의 실체

PIPELINE STEP	MAPPED JAVA CLASSES & COMPONENTS	ROLE
01. 요청 수신	IncidentResponderController / RespondToIncidentUseCase	REST Entry & DTO
02. 관련 정보 조회	IncidentKnowledgePort / IncidentHistoryPort / AlertPolicyPort	Hexagonal Ports
03. Prompt 생성	IncidentPromptFactory	Template Binder
04. AI 분석 연동	IncidentAnalysisService / IncidentChatGateway	Spring AI Gateway
05. 액션 추천 / 반환	IncidentActionRecommendationService / IncidentResponse	Recommendation & Response

메인 Orchestration은 `IncidentResponderService` 가 중심을 잡고 각 Port와 Service를 호출합니다.

⚠ 운영 관점의 문제의식: 왜 아직 부족할까

DEMO-READY ≠ OPS-READY

현재 baseline 코드는 실행 가능하지만, 통제 4축 관점에서 본다면 운영상 위험을 내포합니다.

01. COST RISK

과도한 토큰 소모

필요 이상의 전체 로그와 문서를 무차별 바인딩하여 호출 비용 폭증 위험

02. QUALITY RISK

환각 및 포맷 이탈

검증되지 않은 수식 가설 또는 구조화되지 않은 자연어 답변 이탈 발생

03. ACTION RISK

위험 조치 추천

파괴적 인프라 명령어(재부팅 등)를 검증 및 승인 절차 없이 바로 텍스트 추천

04. FLOW RISK

복기 추적 불가

내부 판단이 어떤 Trace/Span 경로를 타고 생성되었는지 사후 디버깅 불투명

코드 흐름 확인 가이드: IDE 실습 포인트

IDE VERIFICATION CHECKLIST

실습에서 확인할 5개 클래스 축

- **[1] Entry Controller:** API 진입점 — `IncidentResponderController`
- **[2] UseCase DTO:** 입력 객체 — `RespondToIncidentCommand`
- **[3] Core Service:** Orchestration 파이프라인 — `IncidentResponderService`
- **[4] AI Gateway:** Spring AI 호출 연동부 — `IncidentChatGateway`
- **[5] Port/Adapter:** 외부 연동 인터페이스 — `IncidentKnowledgePort`

Read the code as a map

PRACTICE MINDSET

지도를 따라 걷듯이 코드를 읽기

지금부터 진행될 IDE 실습은 작성된 baseline 프로젝트의 구조를 눈으로 검증하는 탐색 과정입니다.

슬라이드에서 확인한 6단계 파이프라인이 실제 메서드 호출 순서와 1:1로 정확히 맞물리는지 확인합니다.

Switch to IDE Practice

오늘 정리 & 다음 수업 안내

TODAY SUMMARY

기준 사례 기반 확립

- **baseline 역할 이해:** 단순 Q&A가 아닌 장애 분석 및 구조화 응답을 만드는 보조 에이전트
- **파이프라인 흐름 수립:** 요청 수신 → 정보 조회 → Prompt 생성 → AI 분석 → 액션 추천 → 응답 반환
- **클래스 매핑 확인:** 추상적 흐름과 실제 Java 클래스 구조간의 1:1 대응 지도 확보

#incident_responder #baseline_architecture #flow_mapping

UP NEXT LECTURE

다음 수업 미리보기

오늘 확인한 baseline 소스 코드 속에 숨어 있는 문제점을 통제 4축 관점으로 본격 리뷰합니다.

LECTURE 6 TOPIC

이 앱에는 어떤 문제가 숨어 있는가 – 샘플 코드를 통제 관점으로 리뷰하기

Controller, Service, ChatClient 호출에서의 문제 지점 파악하기

Move to Next Lecture